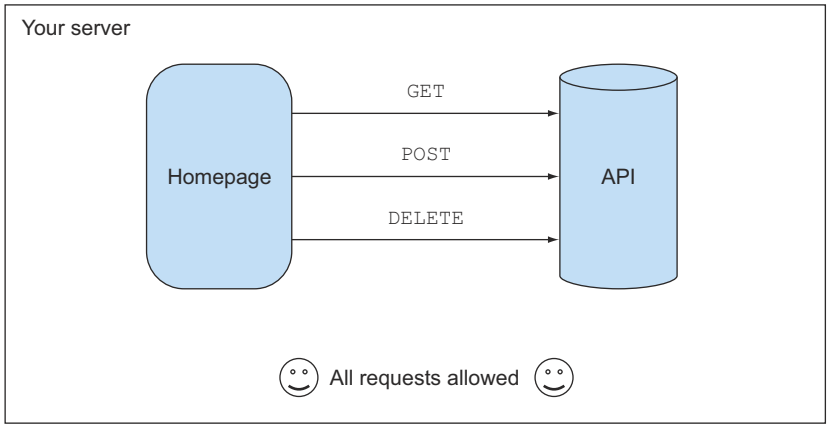


Figure 4.3 Your server, circa 2004. CORS doesn't yet exist, but your site and your API live under the same origin, so they can communicate.

API, as shown in figure 4.3. Because your site lives under the same origin as your API, it can make any type of HTTP request to the API.

Not only does this API fetch news stories, but it also lets you, the owner, edit and delete news items. While you have basic security measures in place, your code never checks which origin a request is coming from, because why should it? All browsers enforce the same-origin policy. There is no such thing as CORS (remember this is 2004), so there is no way for clients from other origins to access your API.

Now fast-forward to 2009. Your news site has become much more popular, and it's still humming along nicely, thanks to the clean architectural separation between your frontend and the API. But then late one night you read that Chrome 4.0 will be released soon, and it supports this new feature called Cross-Origin Resource Sharing that allows cross-origin requests.

You find this a bit troubling, and that night you have nightmares of your server being deluged with all sorts of requests from servers across the web, as shown in figure 4.4. You wake in a cold sweat wondering how you'll protect your server from these cross-origin requests. Will users suddenly be able to send `DELETE` requests without your permission? Why would browsers suddenly break the same-origin policy you have come to rely on?

Luckily, CORS answers these questions. The arrival of CORS didn't cause thousands of server administrators to wake in a cold sweat. In fact, browser support for CORS was a fairly painless rollout because when the CORS spec was being drafted, the spec authors recognized that CORS needed to be introduced in a way that was compatible with existing servers.

The answer to preserving backward compatibility was to introduce the preflight request. The preflight request is a way for the browser to ask the server if it's okay to send a cross-origin request before sending the actual request. The same-origin policy is still